

C Programming

Unit I — Revision Sheet

B.TECH 1ST YEAR · 1ST SEMESTER · FORMULA & CONCEPT GUIDE

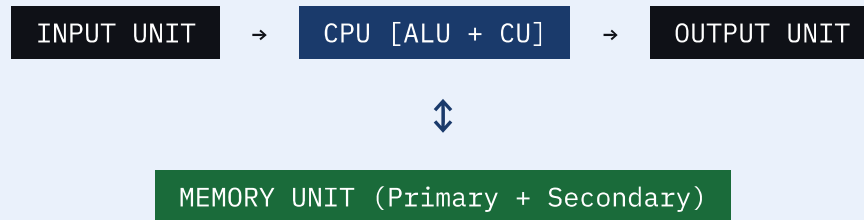
Introduction to
Programming &
Problem Solving

01 History of Computers

Generation	Period	Technology	Features	Examples
1st	1940–1956	Vacuum Tubes	Large, slow, high power consumption, used machine language	ENIAC, UNIVAC
2nd	1956–1963	Transistors	Smaller, faster, less heat; assembly language used	IBM 1401, PDP-1
3rd	1964–1971	Integrated Circuits (IC)	Multiple transistors on a chip; high-level languages	IBM System/360
4th	1971–present	Microprocessors (VLSI)	Personal computers; GUIs; networks	Intel 4004, PC, Mac
5th	Present+	AI / ULSI	Artificial intelligence, parallel processing, natural language	Supercomputers, AI chips

02 Basic Organization of a Computer

BLOCK DIAGRAM (CONCEPTUAL)



Component	Full Form	Function
ALU	Arithmetic Logic Unit	Performs arithmetic (+, −, ×, ÷) and logic (AND, OR, NOT) operations
CU	Control Unit	Directs operations of processor; fetches, decodes and executes instructions
Input Unit	—	Accepts data from user (keyboard, mouse, scanner)
Output Unit	—	Displays/presents results (monitor, printer, speaker)
Memory Unit	—	Stores data and instructions temporarily (RAM) or permanently (HDD/SSD)
Program Counter	PC	Register that holds address of the NEXT instruction to be executed

03 Introduction to Programming Languages

DEFINITION

A **programming language** is a formal set of instructions and syntax used to communicate with a computer to perform specific tasks.

Language Type	Description	Pros	Cons	Example
Machine Language	Binary code (0s and 1s) directly understood by CPU	Fastest execution; no translation needed	Difficult to write; machine-dependent	01001000 11000011
Assembly Language	Symbolic/mnemonic codes (e.g., MOV, ADD)	Faster than HLL; readable	Machine-dependent; tedious	MOV AX, 5
High-Level Language	English-like syntax, human-readable	Easy to learn; portable	Needs compiler/interpreter	C, Python, Java

04 Basics of a Computer Program

Feature	Algorithm	Flowchart	Pseudocode
Form	Step-by-step written procedure	Graphical representation	Informal code-like description
Language	Natural language	Standard symbols (dia tool)	Mix of English + code syntax
Readability	Easy for beginners	Very visual	Closest to actual code
Tool	Pen & paper	Dia, Lucidchart, Visio	Any text editor
Example use	Planning steps	Visualising logic flow	Draft before coding

ALGORITHM EXAMPLE – ADD TWO NUMBERS

- ▶ Step 1: Start
- ▶ Step 2: Input A, B
- ▶ Step 3: SUM = A + B
- ▶ Step 4: Print SUM
- ▶ Step 5: Stop

PSEUDOCODE EXAMPLE

```
BEGIN
  READ a, b
  sum ← a + b
  PRINT sum
END
```

FLOWCHART SYMBOLS

Symbol	Shape	Meaning
Terminal	Oval / Rounded rect	Start / Stop
Process	Rectangle	Computation / Assignment
Decision	Diamond	Yes/No condition
I/O	Parallelogram	Input / Output
Flow Line	Arrow	Direction of flow

05 Compilation & Execution Process



Stage	Role	Tool/File
Pre-processing	Handles <code>#include</code> , <code>#define</code> , macros; removes comments	cpp (C Pre-Processor)
Compilation	Converts C code → assembly; checks syntax errors	cc1 / gcc
Assembly	Converts assembly → machine code (object file)	as (assembler)
Linking	Combines object files + libraries → executable	ld (linker)
Loading & Execution	Loads executable into RAM; CPU runs it	OS loader

06 Data Types, Variables & Constants

Type	Keyword	Size (bytes)	Range	Example
Integer	<code>int</code>	2 or 4	-32768 to 32767 (2B) / -2^{31} to $2^{31}-1$ (4B)	<code>int x = 5;</code>
Float	<code>float</code>	4	$\pm 3.4 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ (~6 sig. digits)	<code>float f = 3.14;</code>
Double	<code>double</code>	8	$\pm 1.7 \times 10^{-308}$ to $\pm 1.7 \times 10^{308}$ (~15 sig. digits)	<code>double d = 3.14159;</code>
Character	<code>char</code>	1	-128 to 127 (signed) / 0 to 255 (unsigned)	<code>char c = 'A';</code>
Void	<code>void</code>	0	No value	<code>void main()</code>

VARIABLE RULES

- ▶ Must start with letter or underscore
- ▶ Can contain letters, digits, underscore
- ▶ Case-sensitive (`age` $\neq$ `Age`)
- ▶ Cannot be a keyword
- ▶ Declaration: `int age;`
- ▶ Initialization: `int age = 20;`

CONSTANTS

- ▶ **Literal:** `int x = 10;` (10 is a constant)
- ▶ **#define:** `#define PI 3.14`
- ▶ **const keyword:** `const int MAX = 100;`

07 Basic Input & Output

PRINTF() – OUTPUT

- **Syntax:** `printf("format string", vars);`
- **Example:** `printf("Age = %d", age);`

SCANF() – INPUT

- **Syntax:** `scanf("format", &var);`
- **Example:** `scanf("%d", &age);`

Format Specifier	Data Type	Example
<code>%d</code>	int	<code>printf("%d", 5);</code>
<code>%f</code>	float	<code>printf("%f", 3.14);</code>
<code>%lf</code>	double	<code>scanf("%lf", &x);</code>
<code>%c</code>	char	<code>printf("%c", 'A');</code>
<code>%s</code>	string	<code>printf("%s", "Hi");</code>

08 Operators in C

Operator Type	Symbols	Meaning	Example
Arithmetic	<code>+ - * / %</code>	Basic math operations	<code>a + b</code> , <code>a % b</code>
Relational	<code>== != > < >= <=</code>	Compare two values → returns 0 or 1	<code>a > b</code> , <code>a == b</code>
Logical	<code>&& !</code>	Logical AND, OR, NOT	<code>(a>0 && b>0)</code>
Assignment	<code>= += -= *= /=</code>	Assign values	<code>a += 5</code> → <code>a = a+5</code>
Increment/Decrement	<code>++ --</code>	Increase/decrease by 1	<code>a++</code> , <code>--b</code>
Bitwise	<code>& ^ ~ << >></code>	Bit-level operations	<code>a & b</code>
Conditional (Ternary)	<code>?:</code>	Short if-else	<code>a>b ? a : b</code>
sizeof	<code>sizeof()</code>	Returns size in bytes	<code>sizeof(int)</code> → 4

09 Type Conversion & Casting

IMPLICIT CONVERSION (WIDENING)

- Done automatically by compiler
- Smaller type → larger type
- No data loss

```
int a = 5;  
float b = a; // auto: 5 → 5.0
```

char → int → long → float → double

EXPLICIT CASTING (NARROWING)

- Programmer forces conversion
- May lose data (truncation)
- Syntax: `(type) expression`

```
float x = 3.99;  
int y = (int)x; // y = 3
```

10 Problem Solving Techniques

CHARACTERISTICS OF A GOOD ALGORITHM (FIODEU)

- **Finiteness** — Must terminate after a finite number of steps
- **Input** — Has zero or more well-defined inputs
- **Output** — Has one or more well-defined outputs
- **Definiteness** — Every step must be clear and unambiguous
- **Effectiveness** — Every step must be feasible (doable)
- **Uniqueness** — Same input always gives same output

11 Problem Solving Strategies

Feature	Top-Down Approach	Bottom-Up Approach
Direction	Problem → Sub-problems	Sub-modules → Full system
Starts with	Main problem / high level	Smallest module / low level
Technique	Divide and conquer; stepwise refinement	Integration; building blocks
Testing	Top modules first; stubs used	Bottom modules first; drivers used
Advantage	Clear structure; easier to manage	Reusable modules; easy integration
Disadvantage	May miss low-level issues early	Overall design may be unclear initially
Used in	Software design, functional decomposition	Object-oriented design, embedded systems

12 Time & Space Complexity

TIME COMPLEXITY

- Measures how execution *time* grows with input size n
- Helps compare efficiency of algorithms
- Expressed using **Big-O notation**

SPACE COMPLEXITY

- Measures how much *memory* an algorithm uses relative to input
- Includes: input space + auxiliary space
- Also expressed with Big-O

Notation	Name	Meaning	Example
$O(1)$	Constant	Always same time regardless of input	Array element access
$O(\log n)$	Logarithmic	Grows very slowly	Binary search
$O(n)$	Linear	Grows proportionally	Linear search
$O(n \log n)$	Linearithmic	Slightly faster than quadratic	Merge sort
$O(n^2)$	Quadratic	Doubles $\rightarrow 4\times$ time	Bubble sort
$O(2^n)$	Exponential	Very slow; impractical for large n	Recursive Fibonacci

BIG-O RULE OF THUMB

Best $\rightarrow$ Worst: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

13 Key Terms & Definitions

Algorithm	Step-by-step procedure to solve a problem	Variable	Named memory location that stores a value
Compiler	Translates entire source code to machine code	Interpreter	Translates and executes code line by line
Flowchart	Graphical representation of an algorithm	Pseudocode	Informal code-like description of an algorithm
Data Type	Defines the kind and size of data a variable holds	Constant	A fixed value that cannot change during execution
Preprocessor	Handles directives before compilation (#include, #define)	Linker	Combines object files and library files into executable
Loader	Loads the executable program into memory for execution	Source Code	Human-written instructions in a programming language
Machine Code	Binary instructions directly executed by CPU	Object Code	Compiled binary (not yet linked) — intermediate file
RAM	Volatile primary memory where programs run	ROM	Non-volatile memory with permanent instructions
ALU	Part of CPU that performs arithmetic and logic operations	Control Unit	Manages fetch-decode-execute cycle of CPU
Program Counter	Register storing address of next instruction	Big-O Notation	Mathematical notation to describe algorithm efficiency
Type Casting	Explicit conversion of one data type to another	Syntax Error	Violation of the grammar rules of a programming language
Runtime Error	Error that occurs during program execution	Logic Error	Program runs but produces incorrect output
Identifier	Name given to variables, functions, arrays, etc.	Keyword	Reserved word with predefined meaning in C (e.g., int, for)
Operator	Symbol that performs an operation on operands	Operand	Value on which an operator acts
Expression	Combination of operands and operators	Statement	Smallest executable unit in C; ends with semicolon
Function	Block of reusable code that performs a specific task	printf()	Library function used to display output on screen
scanf()	Library function used to take input from keyboard	Top-Down Design	Breaking main problem into smaller sub-problems
Bottom-Up Design	Building system by combining small reusable modules	Microprocessor	CPU on a single chip; 4th gen technology



14 Key Concept & Formula Box

COMPLEXITY QUICK REFERENCE

$O(1)$	$O(\log n)$	$O(n)$	$O(n^2)$	$O(2^n)$
Constant	Logarithmic	Linear	Quadratic	Exponential
Best	Great	Good	Slow	Worst

C SYNTAX PATTERNS

Pattern	Syntax
Variable Declaration	<code>data_type variable_name;</code>
Variable Init	<code>int a = 5;</code>
Constant (#define)	<code>#define PI 3.14</code>
Constant (const)	<code>const int MAX = 100;</code>
Output	<code>printf("format", var);</code>
Input	<code>scanf("format", &var);</code>
Type Cast	<code>(type) expression</code>
Ternary Operator	<code>cond ? val_if_true : val_if_false</code>
sizeof	<code>sizeof(data_type)</code>

15 Flashcards

Q: What is an algorithm?

A: A finite, step-by-step procedure to solve a problem

Q: What is pseudocode?

A: An informal, code-like description of an algorithm in plain English

Q: What does an interpreter do?

A: Translates and executes code line-by-line at runtime

Q: What is the Program Counter?

A: A register holding the address of the next instruction to execute

Q: What is $O(n^2)$?

A: Quadratic time — e.g., bubble sort with nested loops

Q: What is explicit type casting?

A: Programmer-forced conversion using (type) syntax; may lose data

Q: What does the linker do?

A: Combines object files and library files to create an executable

Q: What is assembly language?

A: Low-level language using mnemonics (MOV, ADD); needs assembler

Q: Size of float in C?

A: 4 bytes; approx 6 significant decimal digits precision

Q: Size of double in C?

A: 8 bytes; approx 15 significant digits precision

Q: What is a flowchart?

A: A graphical diagram using standard symbols to represent an algorithm

Q: What does a compiler do?

A: Converts entire source code into machine code before execution

Q: What is ALU?

A: Arithmetic Logic Unit — performs math and logical operations in CPU

Q: What is $O(1)$ complexity?

A: Constant time — execution time does not depend on input size

Q: What is implicit type conversion?

A: Automatic widening conversion done by the compiler (no data loss)

Q: What is the preprocessor stage?

A: Handles #include, #define macros before actual compilation

Q: What is machine language?

A: Binary (0s and 1s) code directly understood by the CPU

Q: Size of int in C?

A: 2 bytes (16-bit) or 4 bytes (32/64-bit system)

Q: Size of char in C?

A: 1 byte; range -128 to 127 (signed)

Q: Format specifier for int?

A: %d (decimal integer)

Q: Format specifier for float?

A: %f

Q: Format specifier for char?

A: %c

Q: What is top-down design?

A: Breaking a complex problem into smaller sub-problems recursively

Q: What is bottom-up design?

A: Building a system by combining small, independently-built modules

Q: What is finiteness in an algorithm?

A: The algorithm must terminate after a finite number of steps

Q: What generation introduced microprocessors?

A: 4th generation (1971–present); VLSI technology

Q: What is a variable in C?

A: A named memory location whose value can change during execution

Q: What is a constant in C?

A: A fixed value defined with #define or const that cannot change

Q: What does % operator do?

A: Modulus — returns remainder of integer division (e.g., $7\%3 = 1$)

Q: What is the ternary operator?

A: Shorthand if-else: condition ? value_if_true : value_if_false

Q: Difference between == and =?

A: == is comparison (relational); = is assignment operator

Q: What is ENIAC?

A: First general-purpose computer; 1st generation; used vacuum tubes

16 Exam Tips

⚠ COMMON MISTAKES TO AVOID

- ▶ Confusing **algorithm** (written) with **flowchart** (graphical)
- ▶ Missing `;` at end of C statements
- ▶ Using `=` instead of `==` in comparisons
- ▶ Forgetting `&` in `scanf()`
- ▶ Mixing up `%f` and `%d` format specifiers
- ▶ Ignoring signed vs unsigned range differences
- ▶ Writing overly long answers — keep concise
- ▶ Forgetting compiler $\neq$ interpreter in definitions

✓ WRITING TIPS FOR EXAM

- ▶ **Use diagrams** — flowcharts, block diagrams earn extra marks
- ▶ **Include examples** in every definition answer
- ▶ **Use tables** for comparisons (Top-down vs Bottom-up)
- ▶ Write **short, precise answers** — avoid waffle
- ▶ State **units** for data sizes (bytes, bits)
- ▶ Mention generation years for computer history questions
- ▶ For complexity — always give an example algorithm

🎯 HIGH-PROBABILITY EXAM TOPICS

- ▶ Generations of computers (table)
- ▶ Basic computer organization (diagram)
- ▶ Algorithm vs Flowchart vs Pseudocode
- ▶ Compilation process (flow)
- ▶ Data types with sizes and ranges
- ▶ `printf` / `scanf` syntax + examples
- ▶ Operators table (all types)
- ▶ Implicit vs Explicit type conversion
- ▶ Top-down vs Bottom-up comparison
- ▶ Big-O notation basics

17 Quick Revision Sheet — Last-Day Only

KEY DEFINITIONS

- Algorithm → finite step-by-step procedure
- Variable → named memory location
- Compiler → full code → machine code
- Interpreter → line-by-line execution
- Flowchart → graphical algorithm
- Pseudocode → code-like description
- PC register → next instruction address
- ALU → arithmetic + logic operations
- Preprocessor → #include, #define
- Linker → obj files + libraries = exe

DATA TYPES

- int → 4 bytes, -2^{31} to $2^{31}-1$
- float → 4 bytes, ~6 decimals
- double → 8 bytes, ~15 decimals
- char → 1 byte, -128 to 127
- void → no value
- %d → int | %f → float
- %c → char | %s → string
- scanf needs & before var name
- const / #define → constants
- sizeof(int) = 4 bytes

COMPARISONS

- Top-down → main to sub
- Bottom-up → sub to main
- Machine lang → binary, fastest
- Assembly → mnemonics
- HLL → English-like, portable
- Implicit → auto, no loss
- Explicit → forced, may lose
- $O(1)$ best, $O(2^n)$ worst
- == compare, = assign
- % = modulus (remainder)

OPERATORS

- Arithmetic: + - * / %
- Relational: == != > < >= <=
- Logical: && || !
- Assignment: = += -= *= /=
- Increment: ++ --
- Bitwise: & | ^ ~ << >>
- Ternary: cond ? t : f
- sizeof: returns byte size

GENERATIONS

- 1st → Vacuum tubes (ENIAC)
- 2nd → Transistors (IBM 1401)
- 3rd → IC (IBM 360)
- 4th → Microprocessor (PC)
- 5th → AI / ULSI
- 1st gen: machine language only
- 2nd gen: assembly language
- 3rd gen: high-level languages
- 4th gen: GUIs, networks
- 5th gen: natural language, AI

ALGORITHM PROPERTIES

- Finiteness → must terminate
- Input → 0 or more inputs
- Output → 1+ outputs required
- Definiteness → clear steps
- Effectiveness → doable steps
- Compile steps: pre-process
- → compile → assemble
- → link → load → execute
- Big-O = worst-case complexity
- $O(n \log n)$ = merge sort